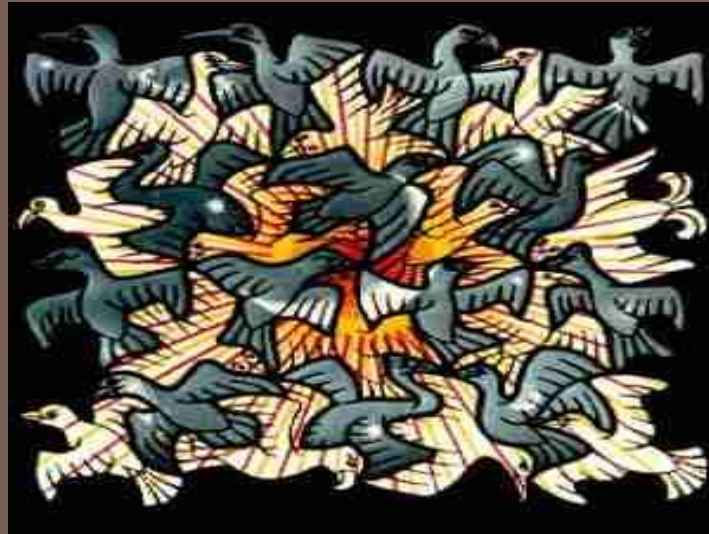


ARQUITETURA E PADRÕES DE SOFTWARE

Prof. Luciano Vale, Msc
lucianovale@unifor.br



PADRÕES GOF — PARTE III

Padrões GoF – parte I

3

- ❑ Singleton
- ❑ Composite
- ❑ Façade
- ❑ Adapter
- ❑ Template Method
- ❑ Strategy
- ❑ Iterator

■	Padrões de Criação
■	Padrões Estruturais
■	Padrões de Comportamento

Padrões GoF – parte II

4

- Prototype
- Decorator
- State
- Observer
- Memento
- Factory Method
- Abstract Factory

■	Padrões de Criação
■	Padrões Estruturais
■	Padrões de Comportamento

Padrões GoF – parte III

5

- ❑ Builder
- ❑ Bridge
- ❑ Proxy
- ❑ Command
- ❑ Chain of Responsibility
- ❑ Mediator
- ❑ Visitor



■	Padrões de Criação
■	Padrões Estruturais
■	Padrões de Comportamento

6

Builder

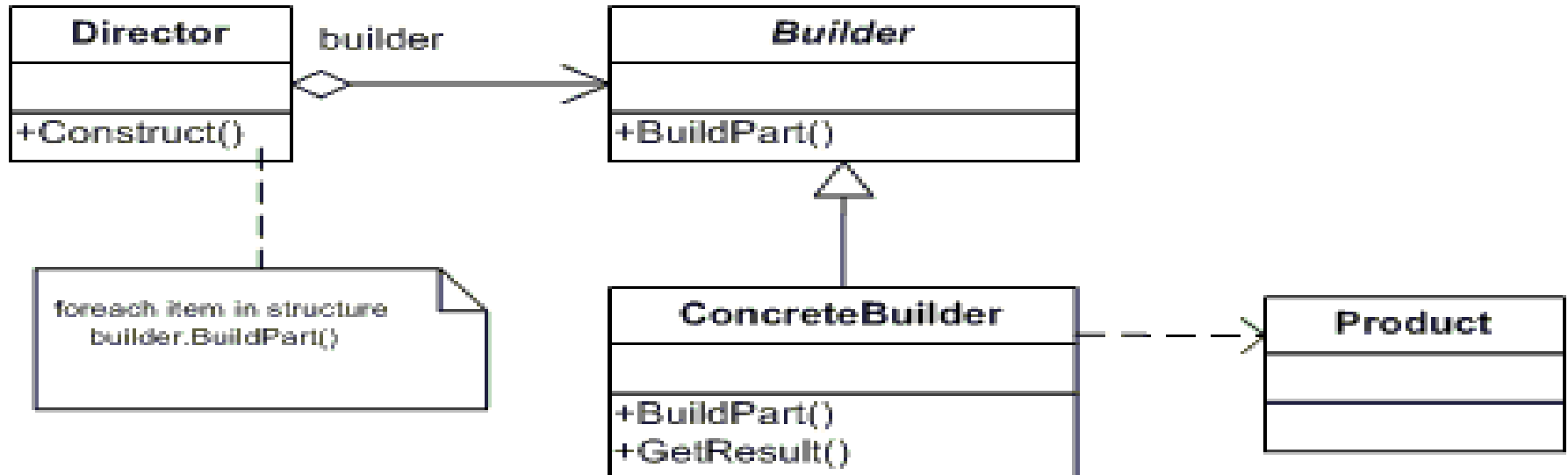
Builder

7

- Intenção: Separar a **construção** de um objeto complexo de sua **representação**, de modo que o **mesmo processo de construção** possa criar **diferentes representações**. (i.e., usar o mesmo processo para construir diferentes tipos de produto.)
 - ▣ GoF Book: “*Builder is intended to decouple the process of building a complex object from the parts that make up the object.*”

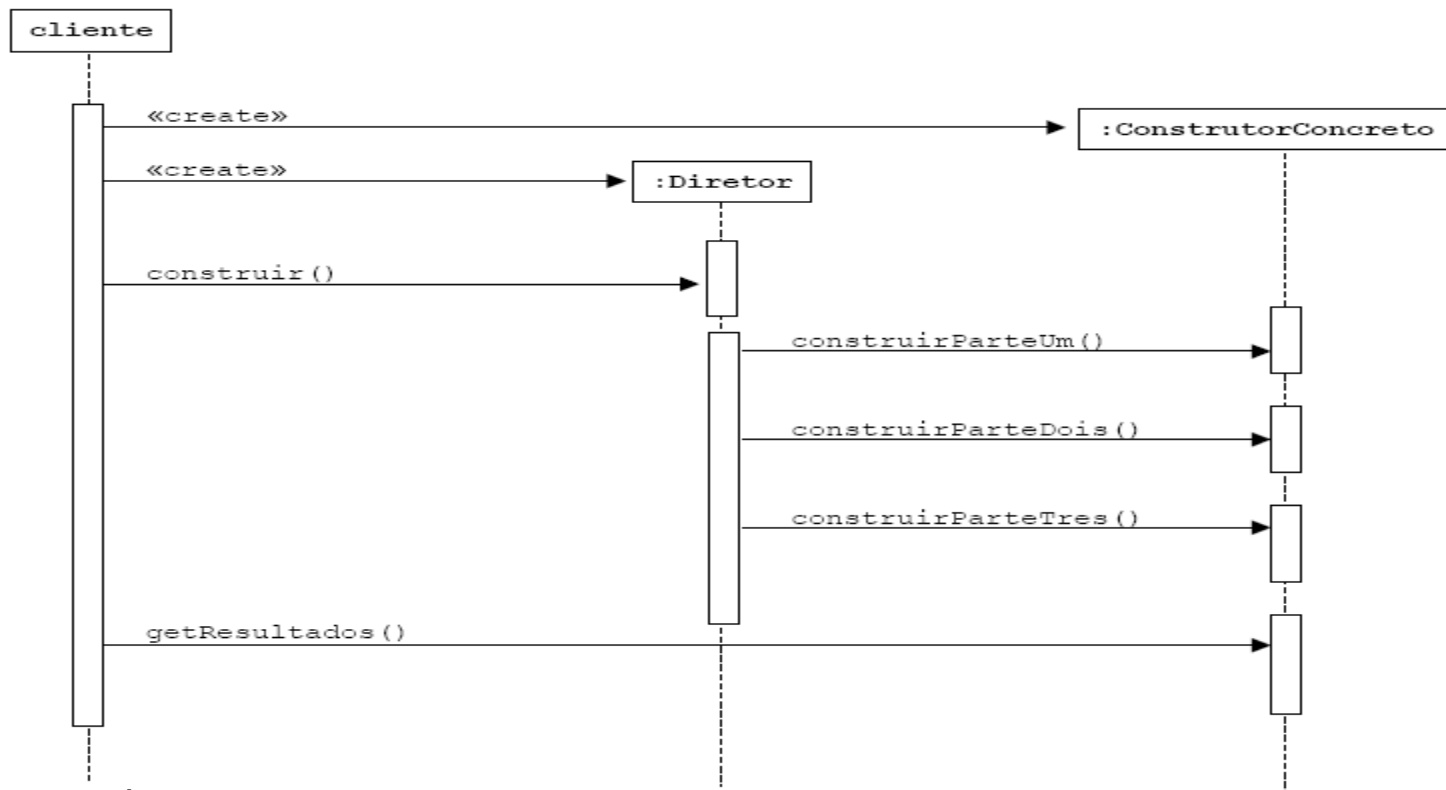
Builder - estrutura

8



Builder - colaborações

9



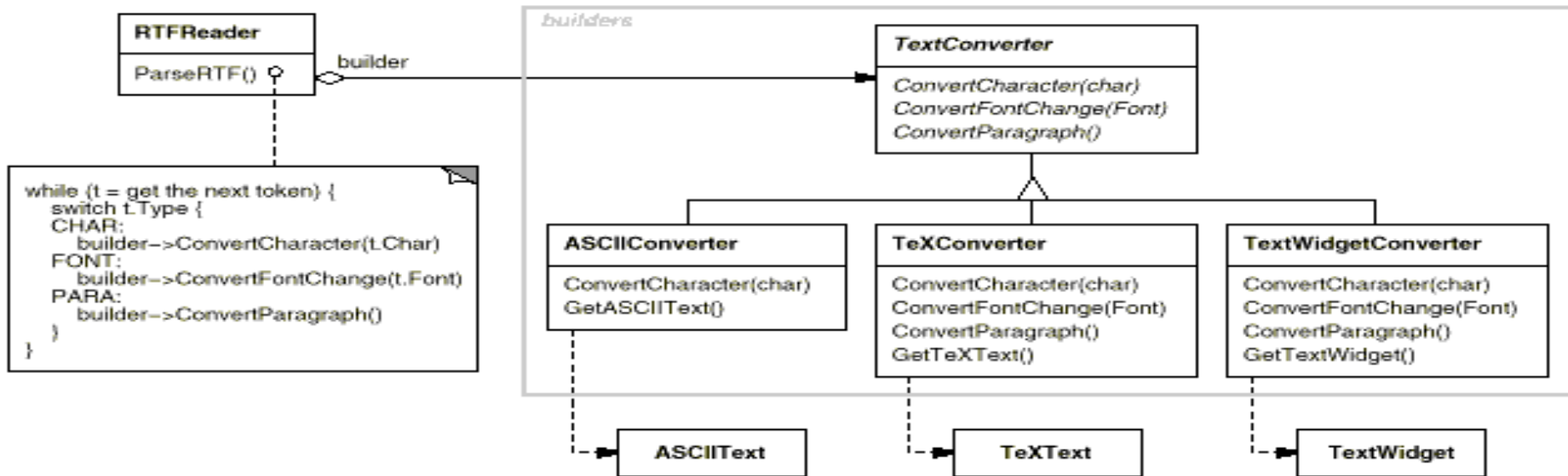
Builder - exemplo (livro GoF)

10

- Programa que converte RTF em outros formatos.
 - ▣ Consiste de um Leitor (Parser) e de um Conversor
- Objetivos:
 - ▣ Dar suporte a diferentes conversões (formatos).
 - ▣ Adicionar um novo formato, sem modificar o leitor.
 - ▣ Separar o Conversor e o Leitor
 - ▣ Reutilizar o algoritmo do Leitor/Parser
- O problema é que a quantidade de conversões em potencial é ilimitada.

Builder - exemplo (livro GoF)

11



Cada classe conversora (subclasses de `TextConverter`) é um **builder** e a classe leitora (`RTFReader`) é o **director**.

Builder - exemplo (livro GoF)

12

- O uso do padrão Builder separa o algoritmo para interpretar um determinado formato de como um arquivo convertido é criado e representado.
 - ▣ Isso permite reusar o algoritmo de *parsing* do RTFReader para criar diferentes formatos (representações) a partir de um documento RTF.
 - ▣ Para isso, basta configurar o RTFReader com diferentes subclasses de TextConverter.

Builder - exemplo 2

13

- <https://www.youtube.com/watch?v=rM0oDoeXgzU>

14

Bridge

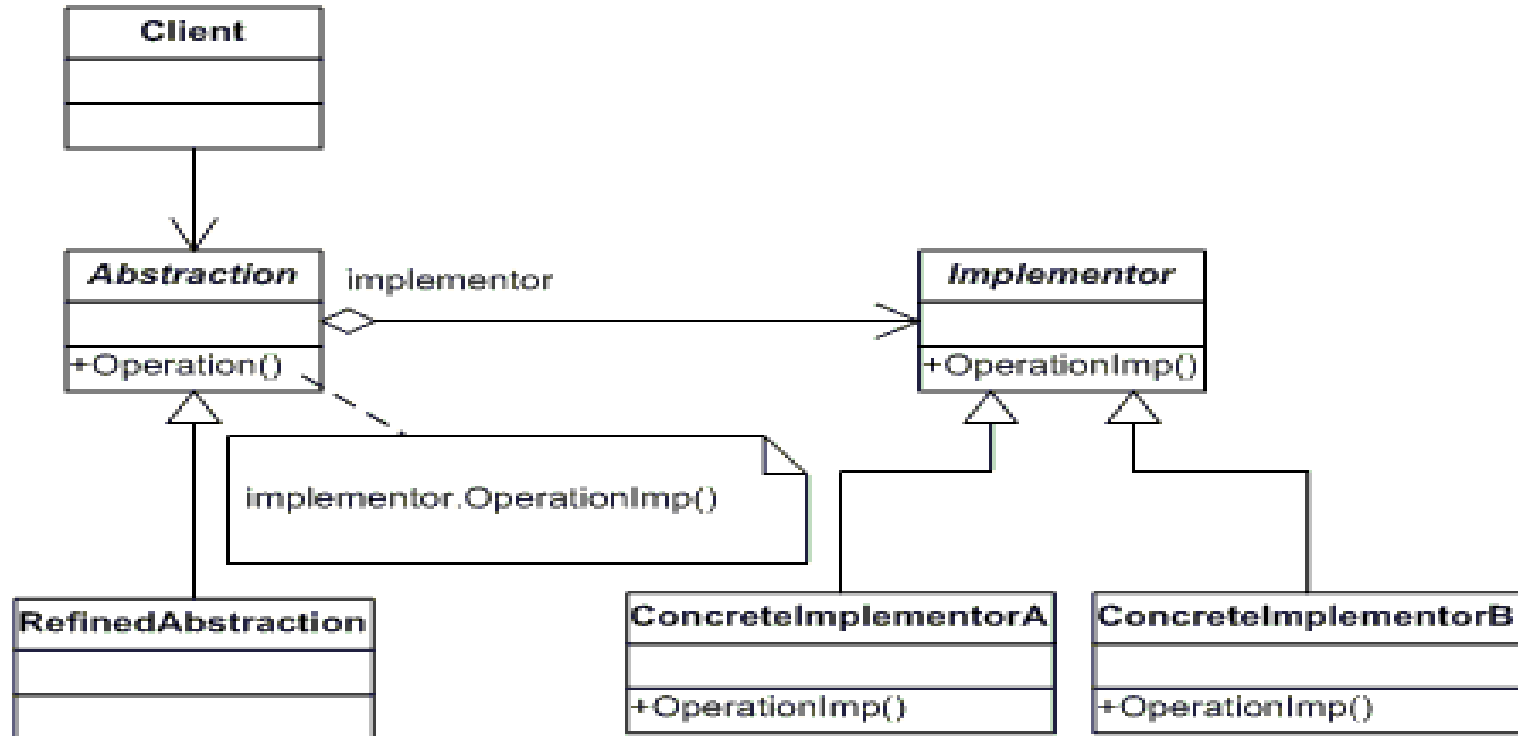
Bridge

15

- Intenção: desacoplar uma abstração de sua implementação, de tal forma que as duas possam variar independentemente uma da outra.
 - ▣ Entenda abstração como uma interface.
- Em linguagens como Java, uma classe que implementa uma interface fica estaticamente acoplada a essa interface (**A** implements **B**).
 - ▣ O padrão Bridge permite eliminar esse acoplamento estático.

Bridge - estrutura

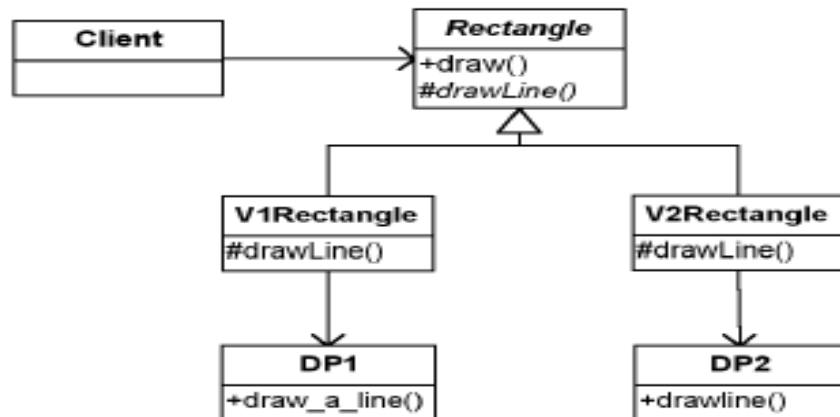
16



Bridge – exemplo (1 / 5)

17

- Considere uma aplicação para desenhar figuras, onde há dois componentes *externos* de desenho a utilizar, DP1 e DP2.
- Primeira versão: “somente retângulos devem ser desenhados”.



Bridge - exemplo (2/5)

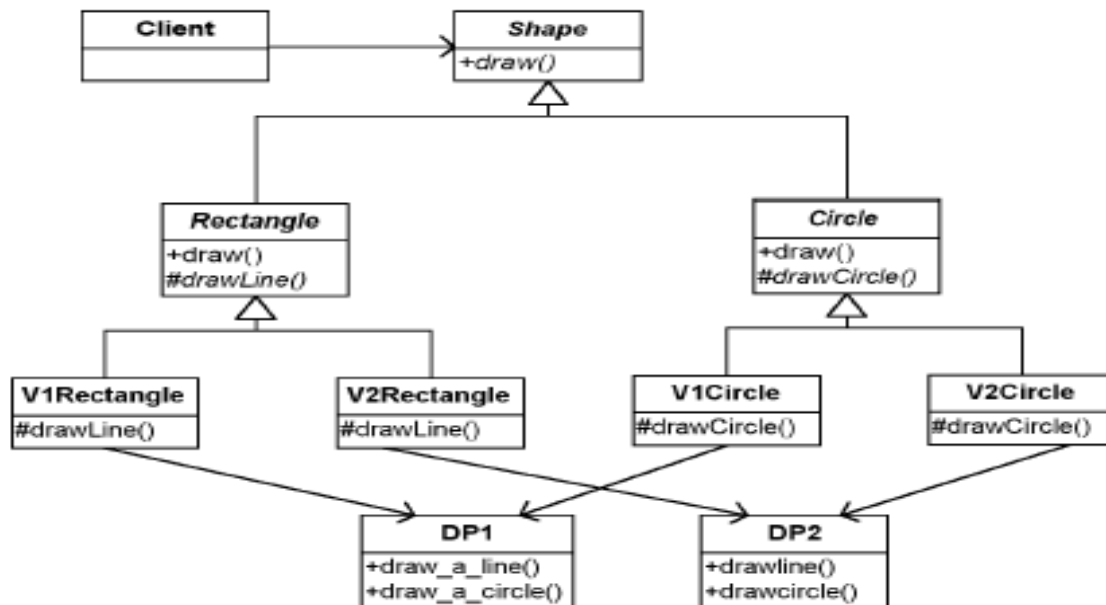
18

- Suponha agora, o seguintes novos requisitos:
 - “As classes externas agora desenhavam círculos. Portanto, o módulo de desenho deve também ter a possibilidade de desenhar círculos”.
 - “Além disso, o cliente do módulo de desenho não precisa saber a diferença entre um retângulo e um círculo.”
- Solução:
 - Criar uma classe abstrata **Shape**, superclasse de **Rectangle** e de **Circle**.
 - O cliente agora se comunica com objetos **Shape**.

Bridge - exemplo (3/5)

19

- A estrutura da nova solução:



Bridge - exemplo (4/5)

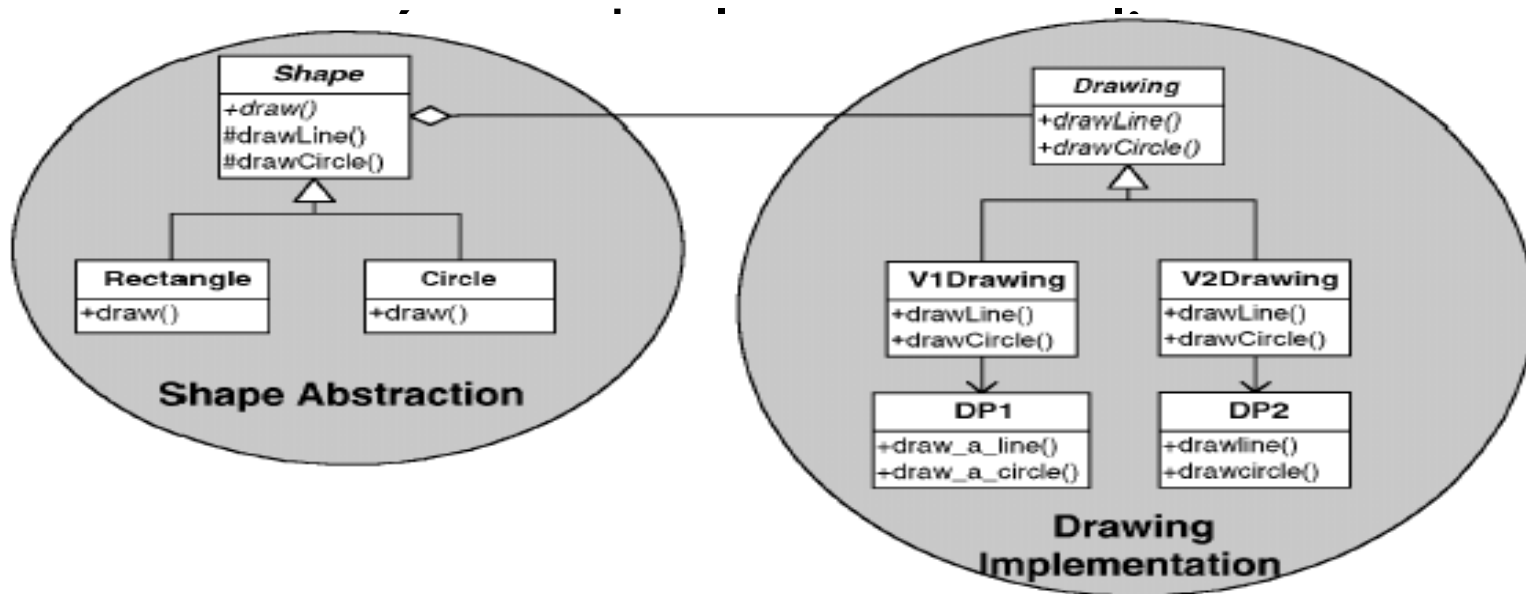
20

- A solução anterior introduz alguns problemas:
 - E se preciso dar suporte a um novo programa de desenho?
 - E se um novo tipo de Figura (Shape) tiver que ser adicionado?
- Em particular, e se a quantidade de tipos de figura e de módulos externos de desenho chegasse à casa das dezenas?
- Resposta: *explosão de classes, manutenção comprometida*
 - A explosão de classes surge porque a abstração (i.e., os subtipos de Shape) e a implementação (i.e., os programas externos de desenho) estão fortemente *acoplados*.
 - i.e., cada tipo de figura (abstração) deve saber que tipo de módulo externo (implementação) deve utilizar.

Bridge - exemplo (5/5)

21

- ❑ Solução: separar (desacoplar) as variações na abstração das variações na implementação, de tal



22

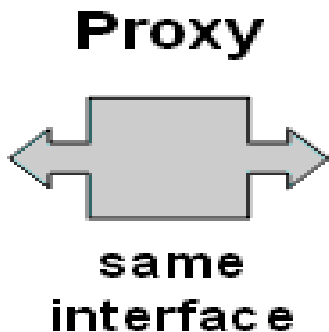
Proxy

Proxy

23

- Intenção: Prover um substituto para um objeto X; através desse substituto, pode-se controlar o acesso a X.

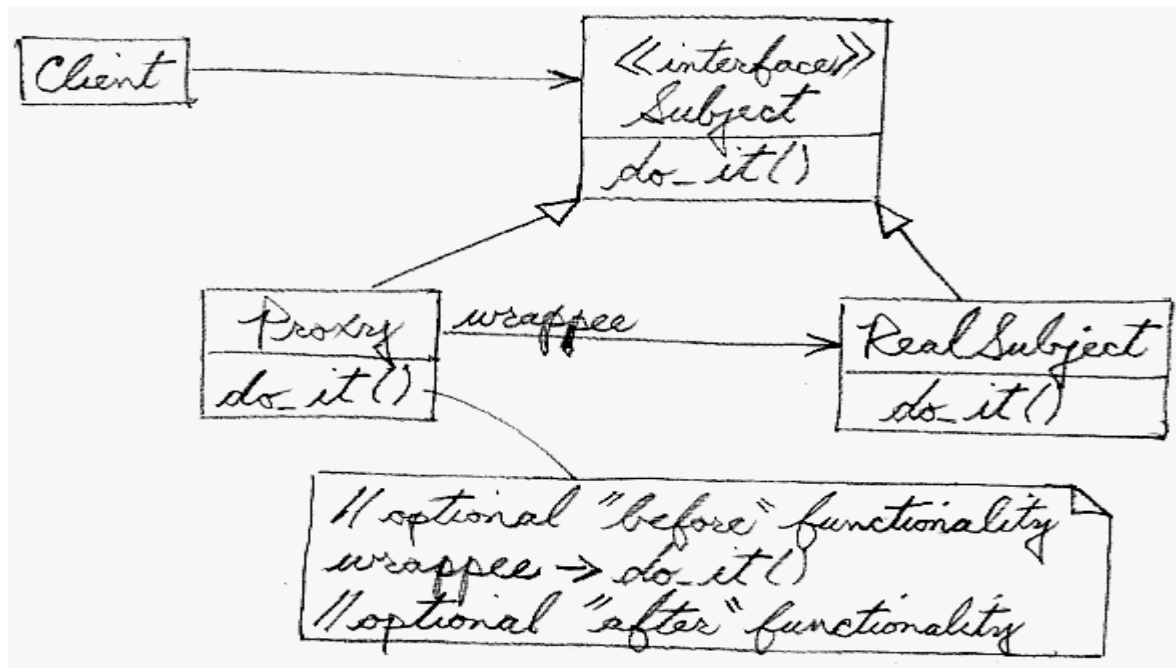
- ▣ GoF Book: *“Provide a surrogate or placeholder for another object*



- 1) Take a class with desirable functionality and wrap it in a new class that provides the same interface and additional functionality (e.g. distributed, controlled, or monitored access).

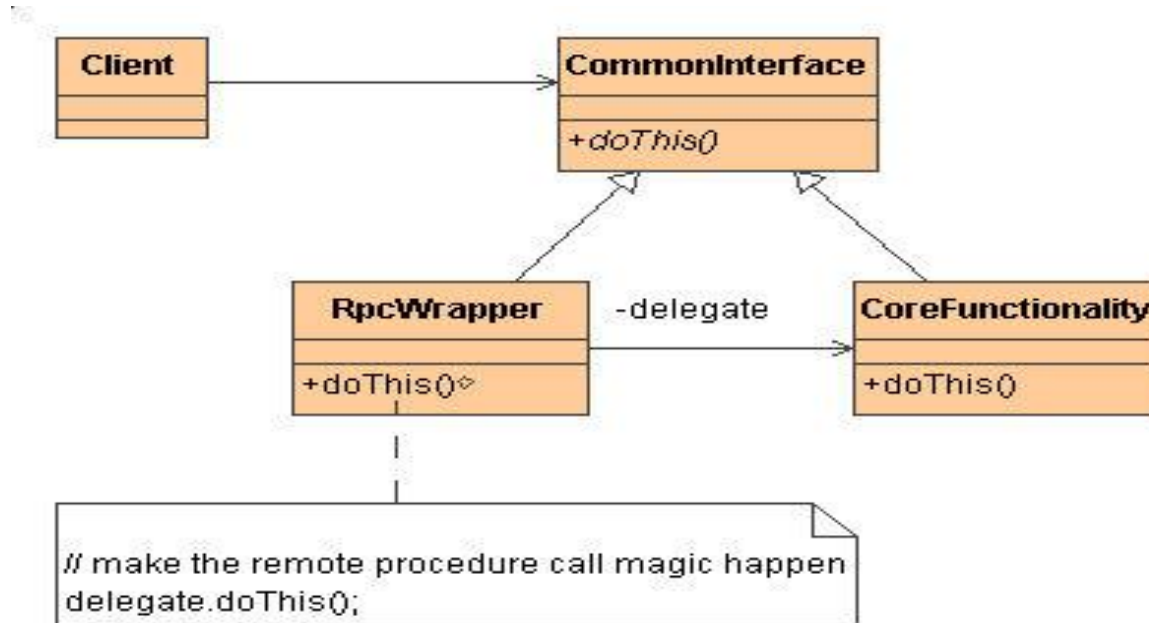
Proxy - estrutura

24



Proxy - exemplo

25



Proxy – exemplo 2

26

- <https://www.devmedia.com.br/conheca-o-pattern-proxy-gof-gang-of-four/4066>

27

Command

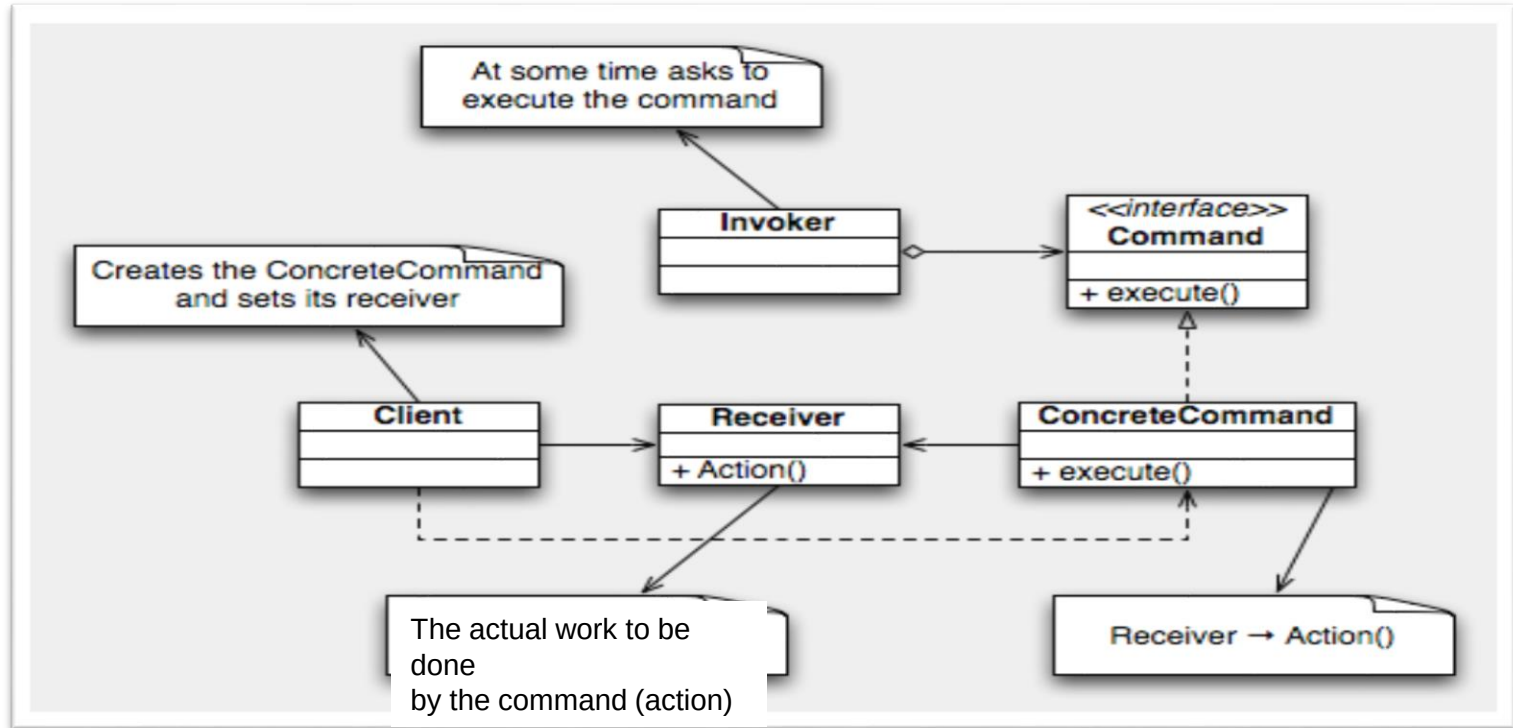
Command

28

- **Intenção:** Representar uma requisição (mensagem) como um objeto, para parametrizar clientes com diferentes requisições, enfileirar (*buffers*) ou fazer registro (*log*) e dar suporte a operações que podem ser desfeitas.
- ▣ *GoF Book: “Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.”*

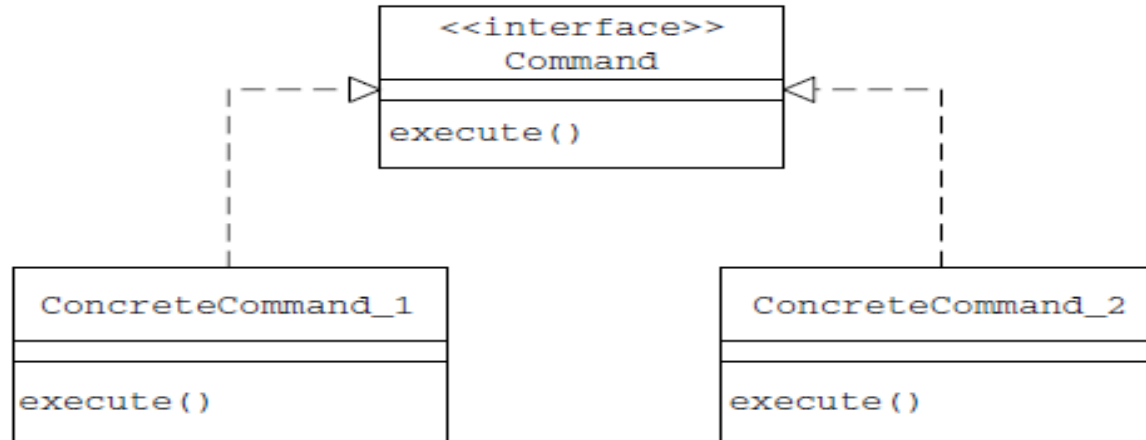
Command - estrutura

29



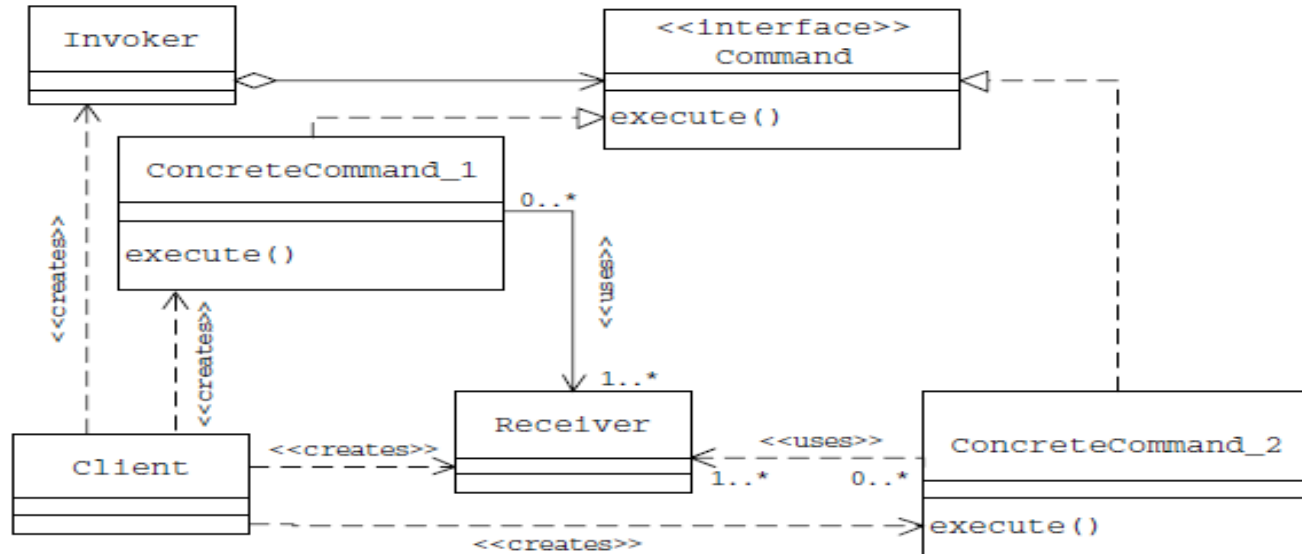
Command – estrutura (cont.)

30



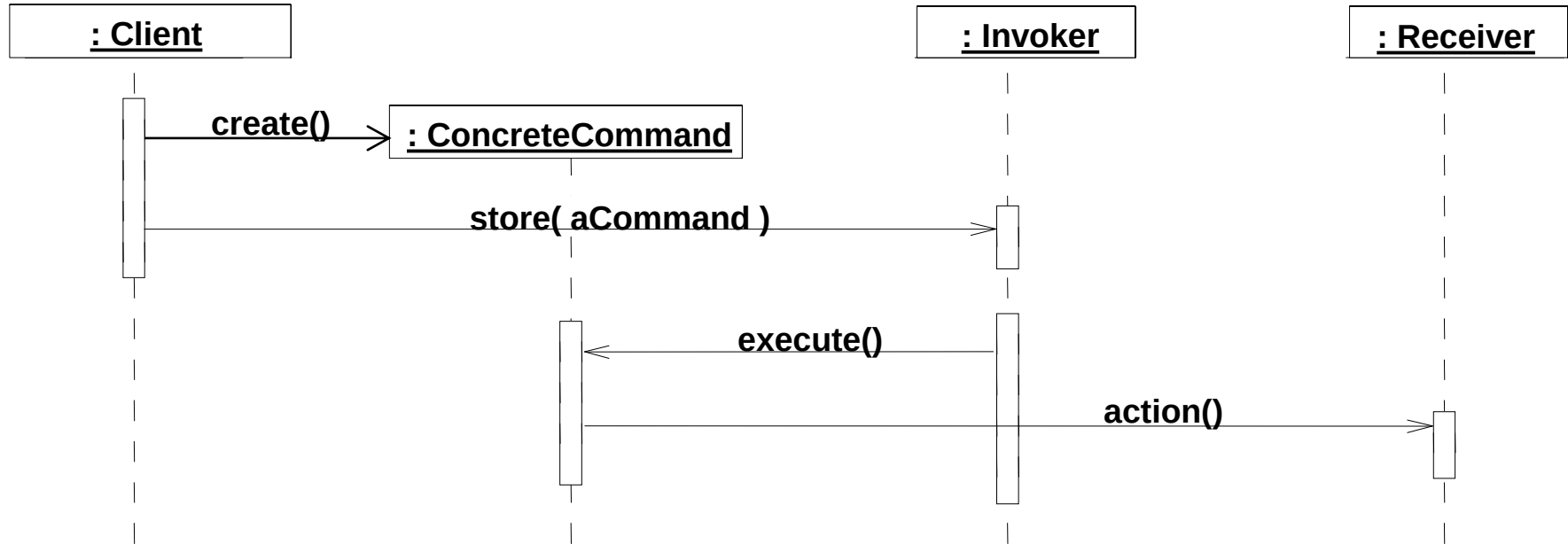
Command – estrutura (cont.)

31



Command - colaborações

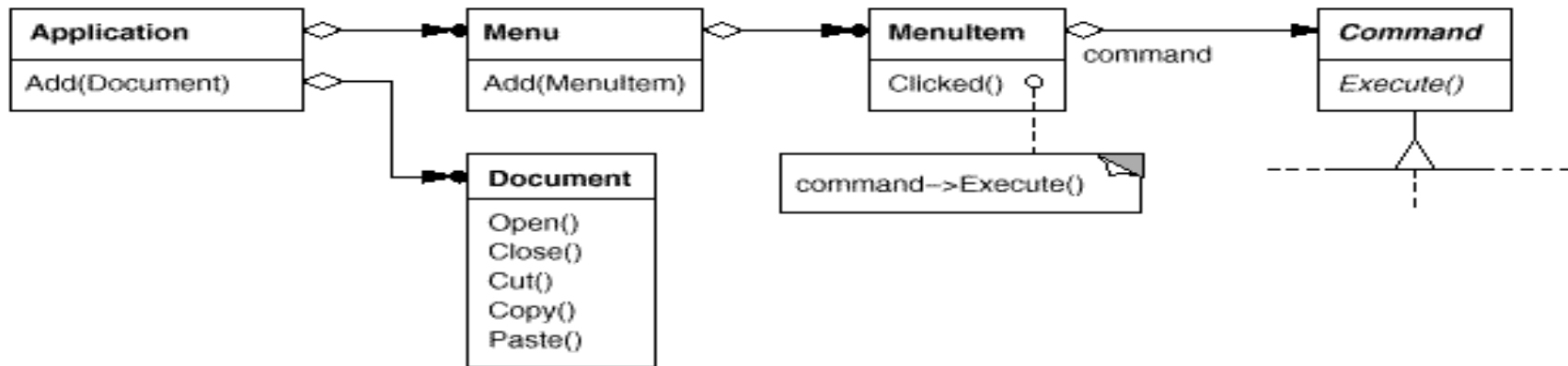
32



Command - exemplo

33

- Um componente de UI pode enviar uma requisição para invocar comportamento específico da aplicação.



Command - exemplo 2

34

- <https://refactoring.guru/pt-br/design-patterns/command/java/example>

35

Chain of Responsibility

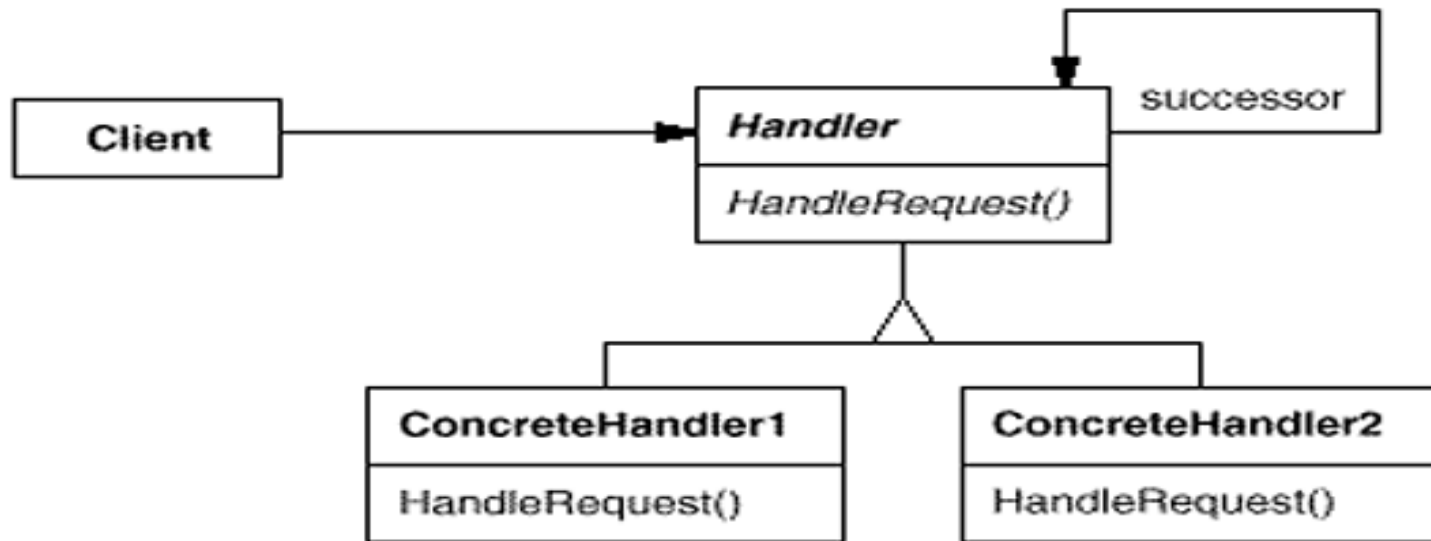
Chain of Responsibility

36

- **Intenção:** Evitar o acoplamento do remetente ao receptor de uma mensagem, ao dar a mais de um objeto a oportunidade de tratar a requisição. Encadear receptores e, através dessa cadeia, passar uma requisição até que um dos objetos da cadeia seja capaz de tratá-la.
- ▣ *GoF Book: “Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.”*

Chain of Responsibility - estrutura

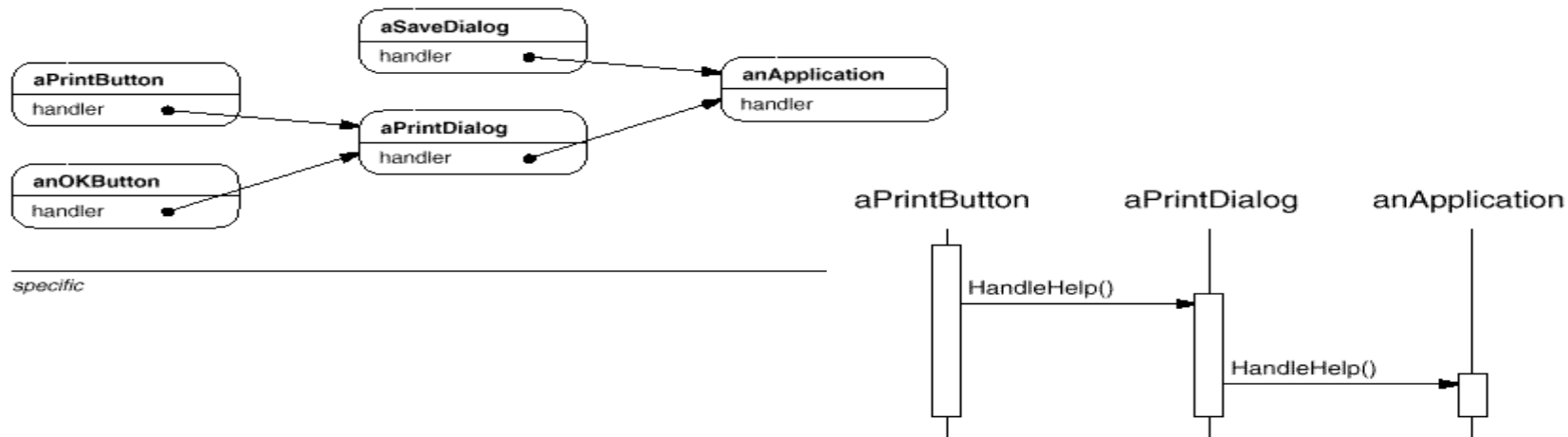
37



Chain of Responsibility - exemplo

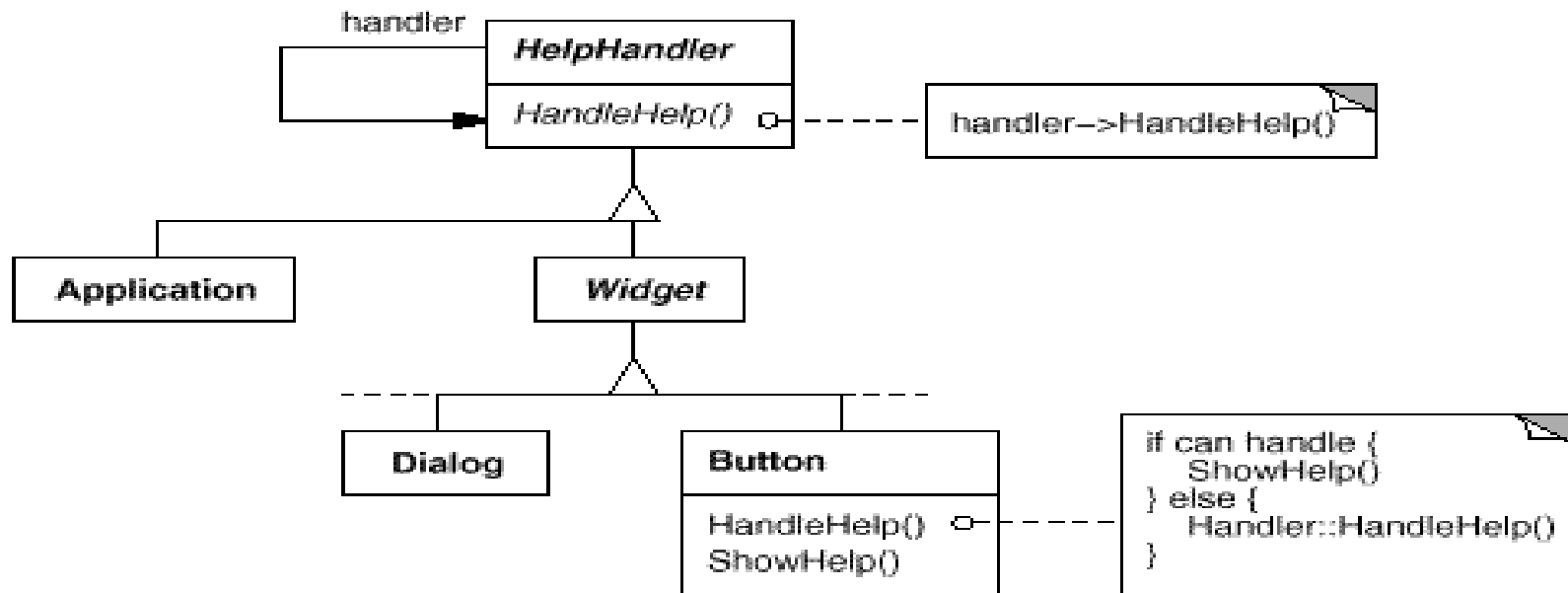
38

- ❑ Recurso de ajuda sensível ao contexto em uma interface gráfica do usuário.



Chain of Responsibility - exemplo

39



Chain of Responsibility – exemplo 2

40

- <https://brizenowordpress.com/category/padroes-de-projeto/chain-of-responsibility/>
- <https://imasters.com.br/desenvolvimento/arquitetura-e-desenvolvimento-de-software-parte-14-chain-responsability>

41

Mediator

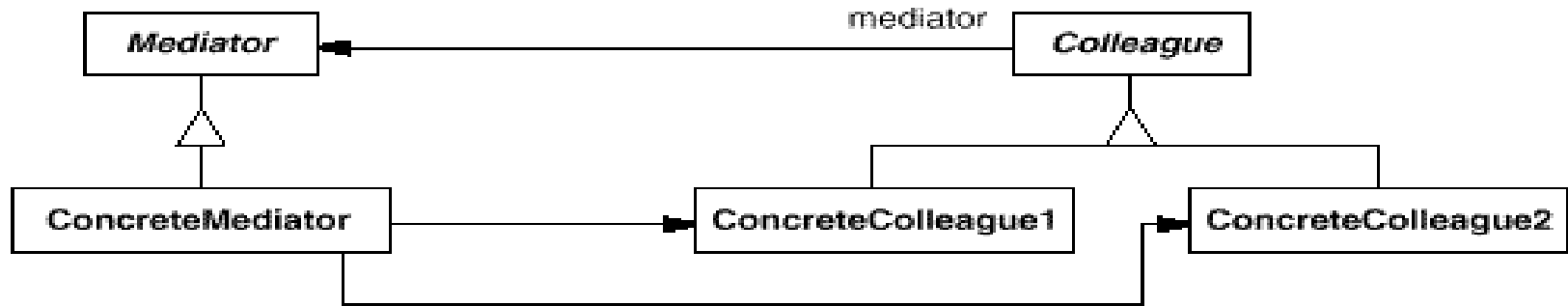
Mediator

42

- Intenção: Definir um objeto que encapsula a forma como um conjunto de objetos interage. Promove o **acoplamento fraco** ao evitar que os objetos façam referências uns aos outros de forma explícita.
- ▣ GoF Book: *“Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently”.*

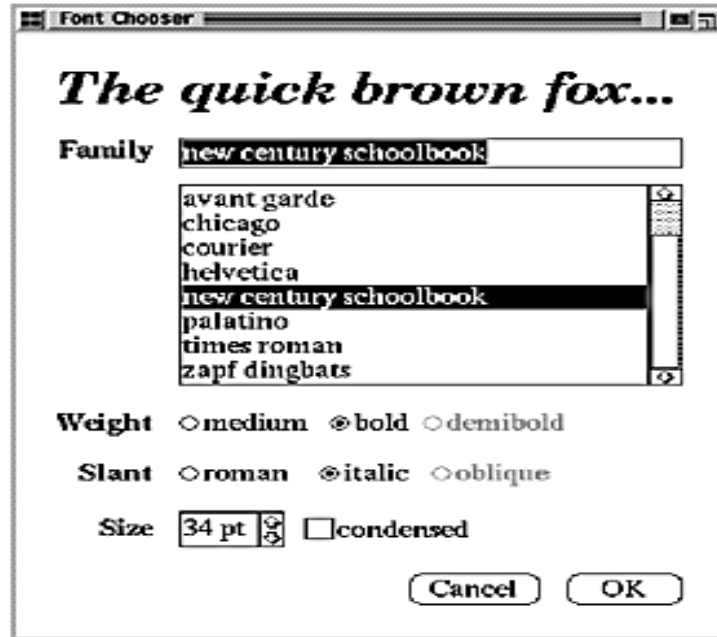
Mediator - estrutura

43



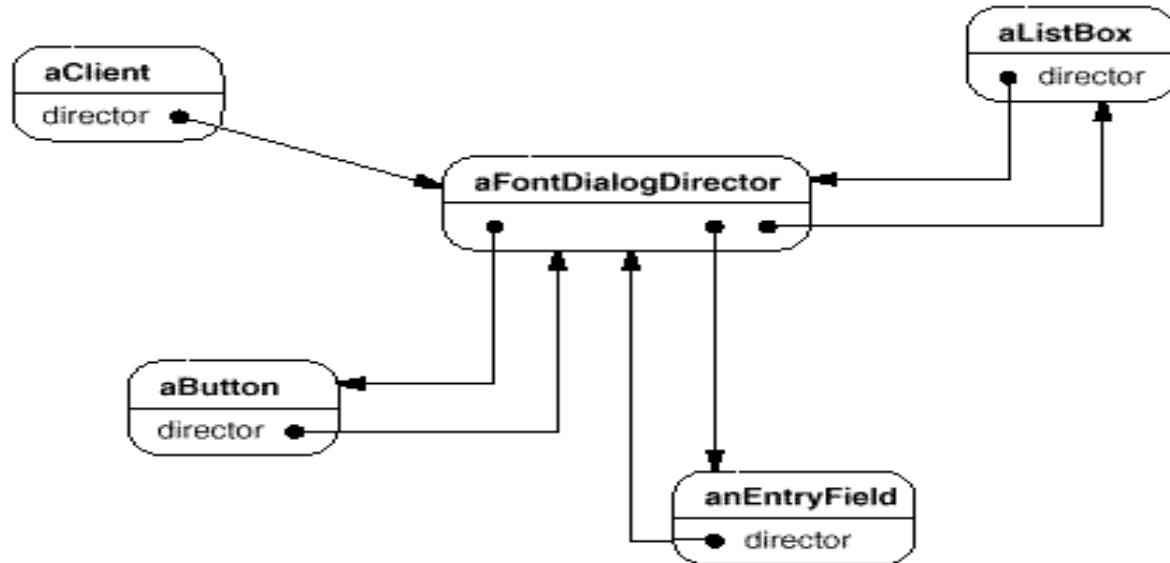
Mediator - exemplo

44



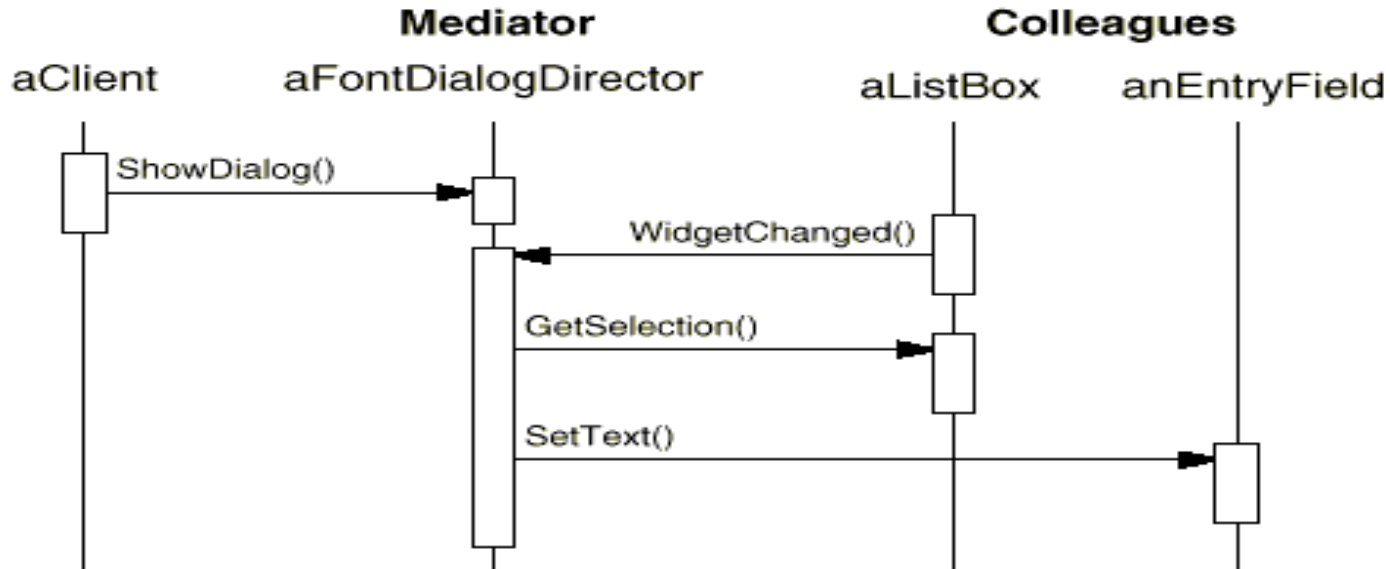
Mediator – exemplo (cont.)

45



Mediator – exemplo (cont.)

46



Mediator – exemplo 2

47

- <https://brizenowordpress.com/category/padroes-de-projeto/mediator/>

Visitor

48

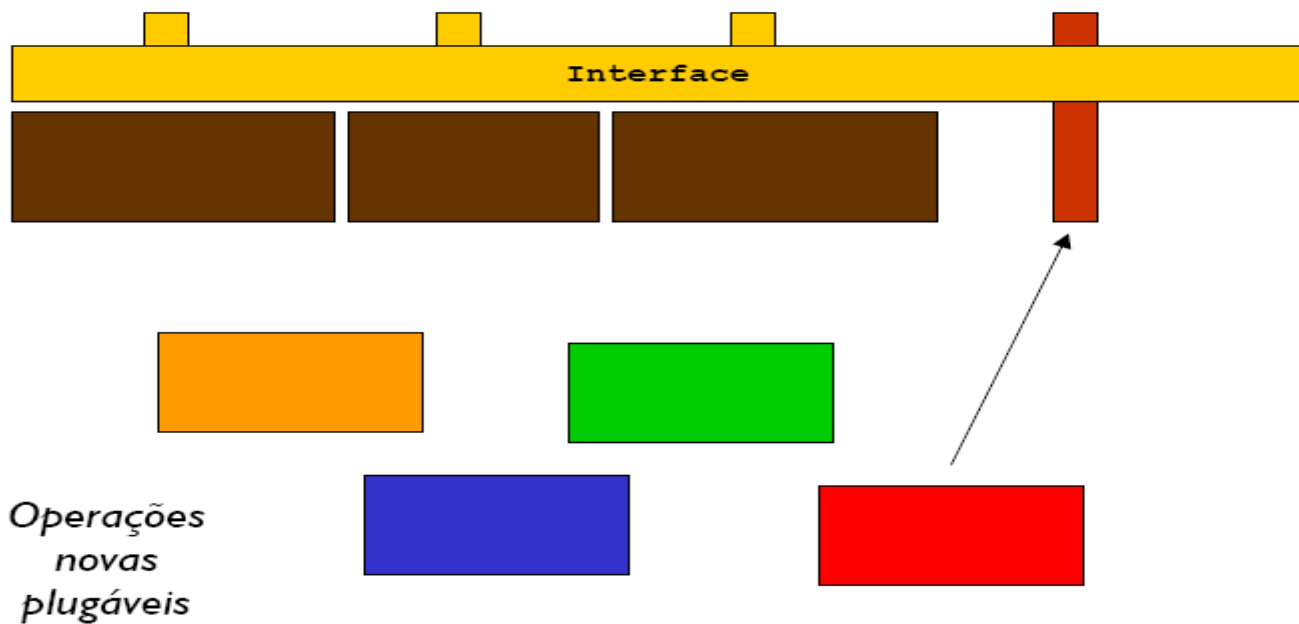
- Intenção: Representar uma operação a ser realizada sobre os elementos de uma estrutura de objetos. Permite definir uma nova operação sem mudar as classes dos elementos sobre os quais ela opera.
- ▣ GoF Book: *“Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.”*

Visitor

49

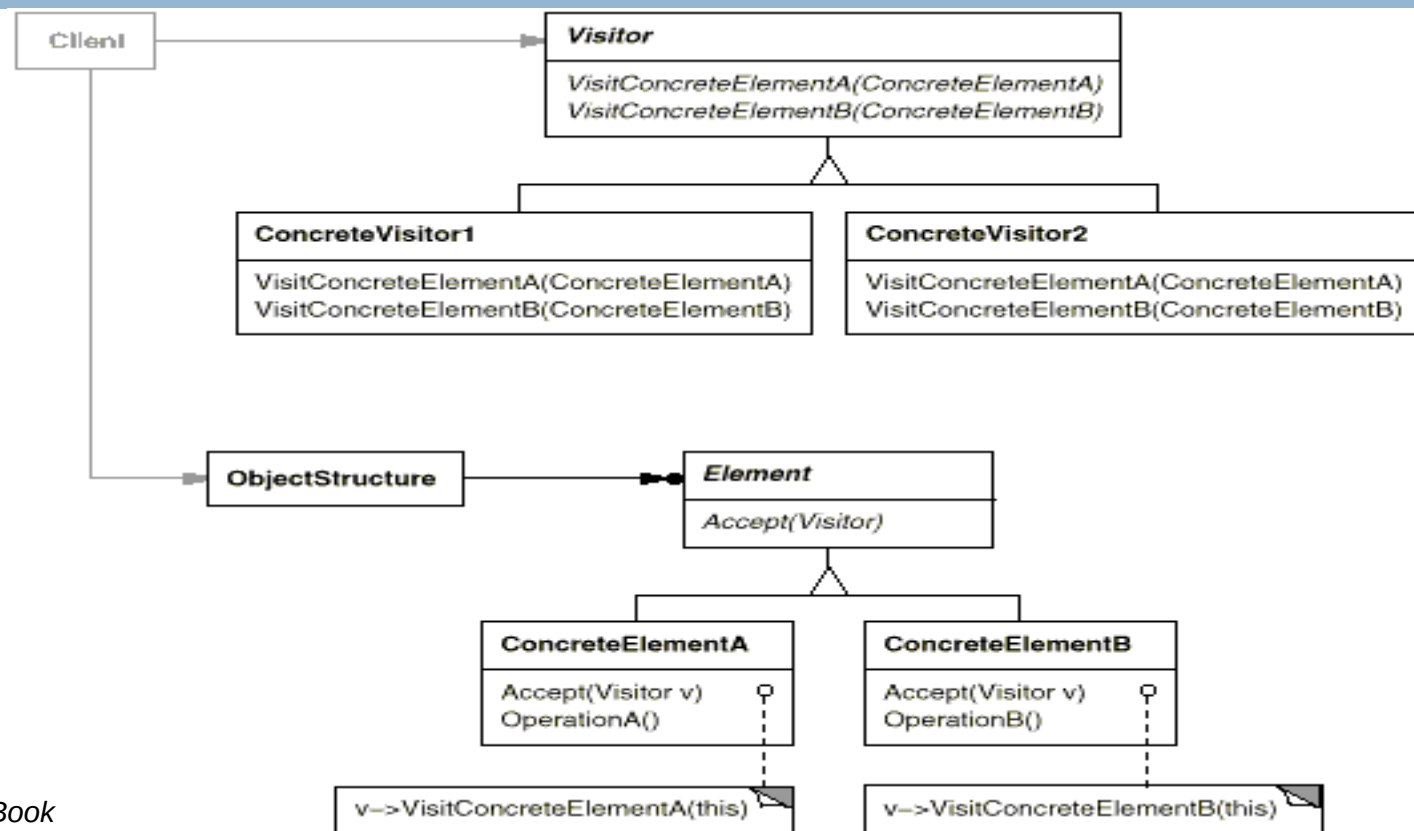
*Operações
fixas*

*Operação de suporte
à extensão*



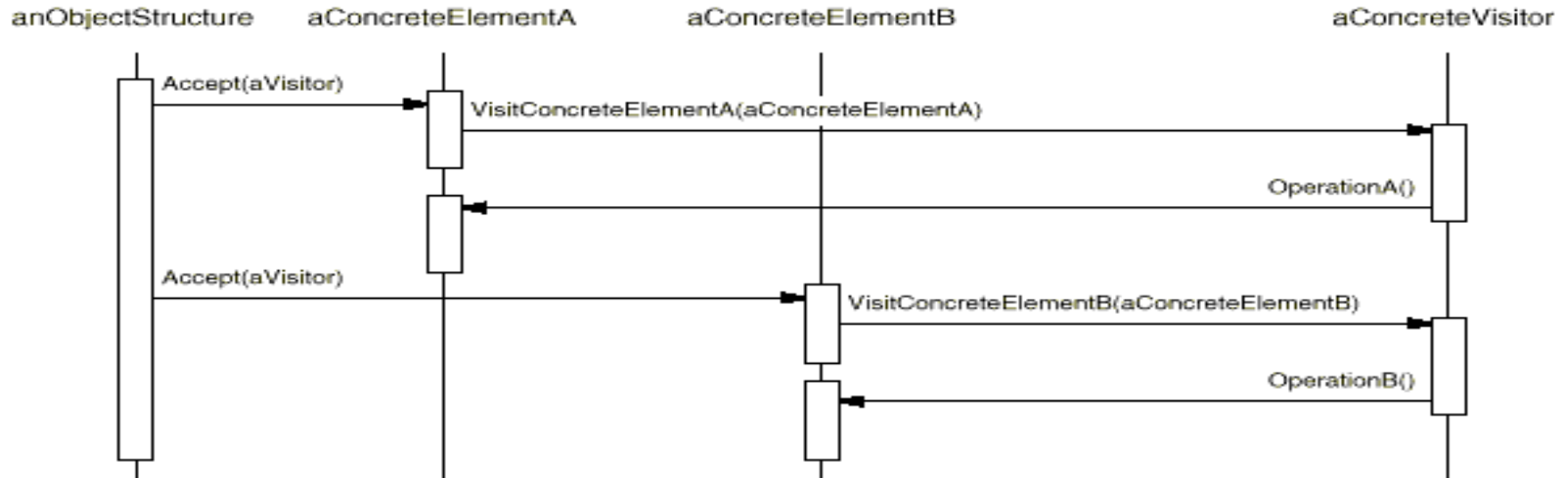
Visitor - estrutura

50



Visitor - colaborações

51



Visitor - exemplo 1

52

- Estrutura de objetos formada pelos seguintes elementos:
 - Vehicle
 - Car
 - Wheel
 - Engine
 - Body

Visitor - exemplo 1

53

```
class Wheel {
    private String name;
    Wheel(String name) {
        this.name = name;
    }
    String getName() {
        return this.name;
    }
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}

class Engine {
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}

class Body {
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}
```

```
abstract class Vehicle {
    protected Engine engine = new Engine();
    protected Body body = new Body();
    protected Wheel[] wheels;

    public Engine getEngine() {
        return this.engine;
    }
    public Body getBody() {
        return this.body;
    }
    public Wheel[] getWheels() {
        return this.wheels;
    }
    public void accept(Visitor visitor) {
        visitor.visitVehicle(this);
    }
}

class Car extends Vehicle {
    public Car() {
        super();
        this.wheels = new Wheel[]{
            new Wheel("front left"),
            new Wheel("front right"),
            new Wheel("back left"),
            new Wheel("back right")
        };
    }
    public void accept(Visitor visitor) {
        visitor.visitCar(this);
    }
}
```

Visitor - exemplo 1

54

```
interface Visitor {  
    void visit(Wheel wheel);  
    void visit(Engine engine);  
    void visit(Body body);  
    void visitCar(Car car);  
    void visitVehicle(Vehicle vehicle);  
}
```

Visitor - exemplo 1

55

```
class PrintVisitor implements Visitor {

    public void visit(Wheel wheel) {
        System.out.println("Visiting "
                           + wheel.getName()
                           + " wheel");
    }
    public void visit(Engine engine) {
        System.out.println("Visiting engine");
    }
    public void visit(Body body) {
        System.out.println("Visiting body");
    }

    public void visitVehicle(Vehicle vehicle) {
        System.out.println("Visiting vehicle");
    }
    public void visitCar(Car car) {
        System.out.println("Visiting car");
        car.getEngine().accept(this);
        car.getBody().accept(this);
        for(Wheel wheel : car.getWheels()) {
            wheel.accept(this);
        }
    }
}
```

```
class DoVisitor implements Visitor {
    public void visit(Wheel wheel) {
        System.out.println("Steering my wheel");
    }
    public void visit(Engine engine) {
        System.out.println("Starting my engine");
    }
    public void visit(Body body) {
        System.out.println("Moving my body");
    }
    public void visitCar(Car car) {
        System.out.println("Starting my car");
        car.getEngine().accept(this);
        car.getBody().accept(this);
        for(Wheel wheel : car.getWheels()) {
            wheel.accept(this);
        }
    }
    public void visitVehicle(Vehicle vehicle) {
        System.out.println("Starting my vehicle");
    }
}
```

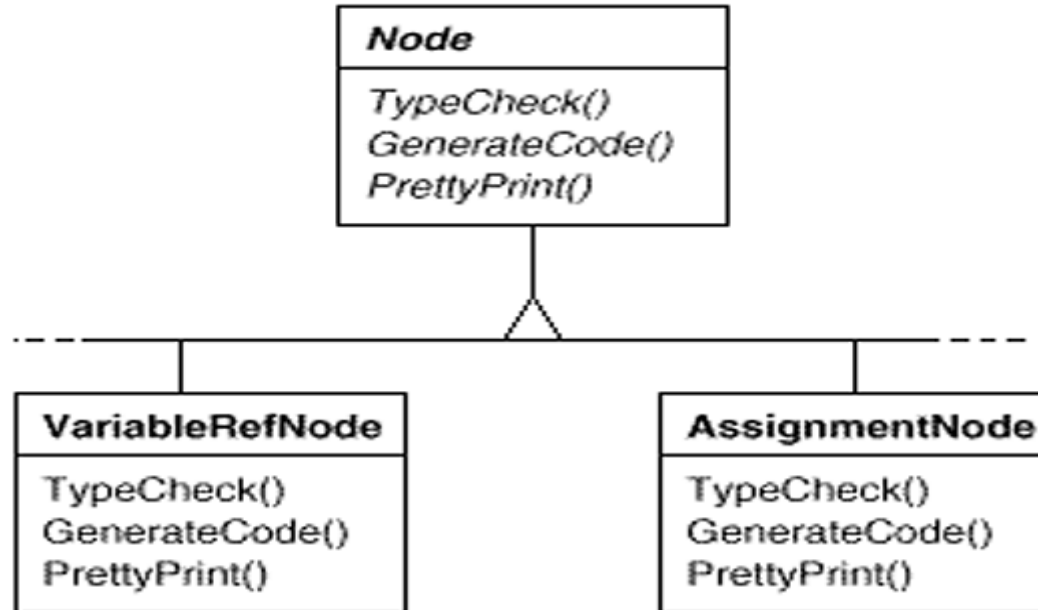
Visitor - exemplo 1

56

```
public class VisitorDemo{
    static public void main(String[] args){
        Car car = new Car();
        Visitor printVisitor = new PrintVisitor();
        Visitor doVisitor = new DoVisitor();
        car.accept(printVisitor);
        car.accept(doVisitor);
    }
}
```

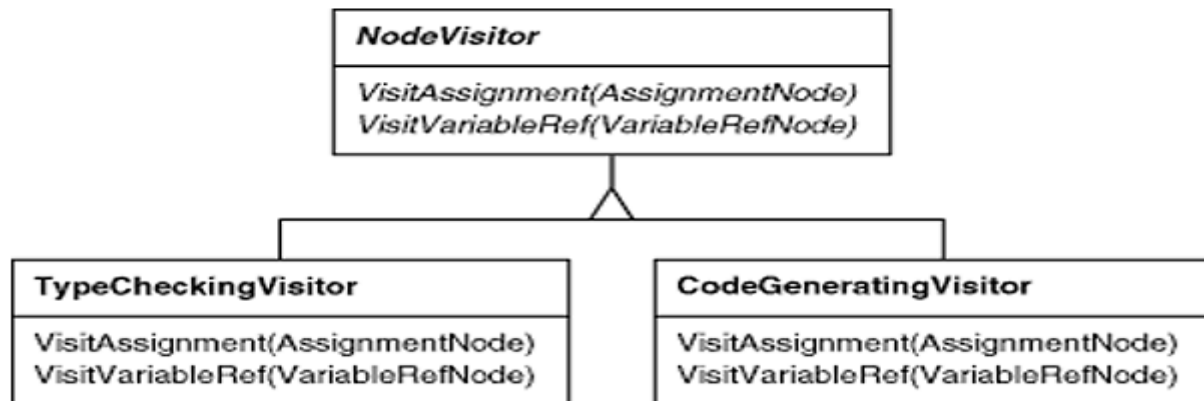
Visitor – exemplo 2

57



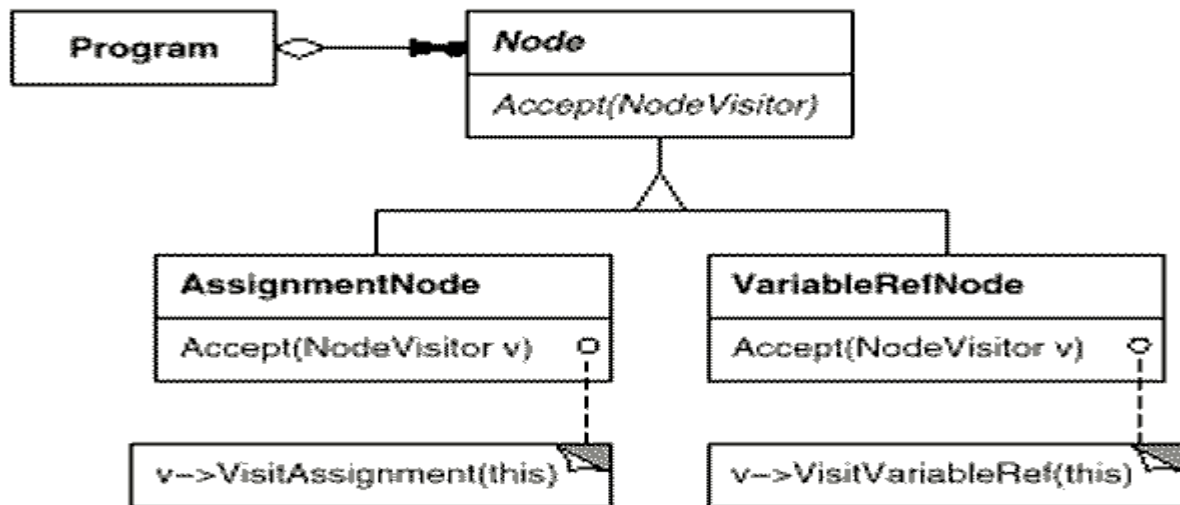
Visitor – exemplo 2

58



Visitor – exemplo 2

59



Referências

60

- Professor Prof. Eduardo Bezerra.
- Gamma et al. ***Padrões de Projeto - Soluções Reutilizáveis de Software Orientado a Objetos***, Porto Alegre: Bookman, 2000.
- Steven Metsker, ***Padrões de Projeto em Java***, Porto Alegre: Bookman, 2004.
- Freeman et al. ***Use a Cabeça!: Padrões de Projetos***, Rio de Janeiro: Alta Books, 2005.

Referências

61

